



1 Types & literals

int i = 42; long n = 9_000L; _ is a digit separator
byte b = 255; short s = -32768; 8- and 16-bit
double d = 3.14; float f = 3.14f; binary floating point
decimal m = 19.99m; exact, use it for money
bool ok = true; char c = 'A'; one UTF-16 unit
int h = 0x2A; int b = 0b101010; hex and binary, both 42
var x = new List<int>(); type inferred at compile time
int n = default; nint p = 42; zero value; native-size
const double Pi = 3.14159; compile-time constant
readonly DateTime _t = DateTime.Now; set once, in a ctor
public required string Name { get; init; }; caller must set it · [C# 11](#)

2 Strings

(\$"{greeting} {name}!") interpolation · [C# 6](#)
@"C:\Users\Docs" verbatim: escapes off, newlines kept
"""{ "a": 1 }""" raw literal, no escaping · [C# 11](#)
\$\$"""{ "n": "{ {x} } " }""" { { } is the hole · [C# 11](#)
ReadOnlySpan<byte> u = "Hi"u8; UTF-8 literal · [C# 11](#)
"\e[31mError\e[0m" \e is the ESC escape · [C# 13](#)
sb.Append(\$"Item {i}, "); StringBuilder in loops
StringComparison.Ordinal always pick a comparison
text.Split(',') · text.Trim() common helpers
string.IsNullOrEmpty(s) null or zero length

3 Null handling

int? age; string? name; nullable value / reference
customer?.Name?.Length whole chain short-circuits
userName ?? "Anonymous" fallback when null
userName ??= "Anonymous"; assign only if null · [C# 8](#)
x is null · x is not null ignores overloaded ==
customer?.Order = GetOrder(); only if non-null · [C# 14](#)
customer?.Balance += 10; compound, not ++ · [C# 14](#)

4 Control flow

if (a) X(); else if (b) Y(); else Z(); if / else chain
age >= 18 ? "Adult" : "Minor" ternary conditional

```
switch (day)
{
    case DayOfWeek.Sunday: Weekend(); break;
    default: Weekday(); break;
}
```

every section needs break or return

```
string Kind(object x) => x switch
{
    Circle c when c.Radius > 10 => "Big circle",
    Rectangle { Width: 0 }      => "Line",
    null                        => "No shape",
    _                           => "Other"
};
```

switch expression: patterns and a when guard · [C# 8](#)

5 Loops

for (int i = 0; i < 10; i++) known iteration count
foreach (string n in names) any IEnumerable
foreach (var (i, n) in xs.Index()) index + item · [.NET 9](#)
foreach (var (k, v) in dict) deconstructs · [C# 7](#)
while (c) { } · do { } while (c); zero times / at least once
continue; · break; next iteration / leave loop
yield return item; lazy iterator method

```
outer: for (int r = 0; r < h; r++)
    for (int c = 0; c < w; c++)
        if (grid[r, c].IsGoal) break outer;
```

labeled break / continue · [C# 15 preview](#)

6 Pattern matching

value is string text type pattern + declaration
p is { Name: "Alice", Age: >= 30 } property pattern
o is { Customer: { Name: "A" } } nested property pattern
x is > 20 and < 30 · x is not null and/or/not · [C# 9](#)
(x, y) is (> 0, > 0) tuple pattern
shape is Circle(var radius) positional, on a record
arr is [var a, .., var z] · [> 0, ..] list pattern · [C# 11](#)
var (name, _) = person; var binds, _ discards

7 Methods, parameters & lambdas

public int Add(int a, int b) => a + b; expression-bodied
void Greet(string n, string hi = "Hi") optional parameter
Greet(hi: "Yo", n: "Al"); named arguments, any order
void Swap(ref int a, ref int b) caller sees the change
bool Try(string s, out int result) assign before return
double D(in Point a, in Point b) read-only ref · [C# 7.2](#)
double L(ref readonly Vector3 v) wants a variable · [C# 12](#)
int Sum(params int[] numbers) variable-length args
params ReadOnlySpan<int> n no array alloc · [C# 13](#)

```
int Factorial(int n) {
    int Calc(int x) => x <= 1 ? 1 : x * Calc(x - 1);
    return Calc(n); }
```

local function, scoped to one method · [C# 7](#)

```
Func<int,int,int> f = (a, b) => a + b; lambda into a delegate
(x, y = 10) => x + y default lambda param · C# 12
```

8 Classes, records & structs

public string Name { get; set; }; auto property
public bool IsAdult => Age >= 18; computed property
public string Id { get; init; }; init-only · [C# 9](#)
class Person(string name, int age) primary ctor · [C# 12](#)
record Person(string F, int Age); value equality · [C# 9](#)
person with { Age = 31 } copy with changes
record struct Point(double X); value-type record · [C# 10](#)
readonly record struct Rgb(byte R); members are get/init
sealed class Cache { } no further inheritance
abstract class · static class must derive / no instances

9 Interfaces & generics

```
public interface ILogger
{
    void Log(string message);
    void LogError(string m) => Log($"ERROR: {m}");
}
```

default interface implementation · [C# 8](#)

```
where T : struct // value      where T : class // ref
where T : notnull // never null
where T : new()   // ctor, must be last
```

constraints stack per type parameter
where T : allows ref struct T can be Span<T> · [C# 13](#)

```
static T Sum<T>(IEnumerable<T> v) where T : INumber<T>
    => v.Aggregate(T.Zero, (a, b) => a + b);
```

generic math over any numeric type · [C# 11](#)
static abstract TSelf Create(); on an interface · [C# 11](#)

10 Extension members

```
public static string Truncate(this string s, int n)
    => s.Length <= n ? s : s[..n];
```

classic extension method, still valid, same IL

```
extension<T>(IEnumerable<T> source)
{
    public bool IsEmpty => !source.Any();
}
```

extension property, in a top-level static class · [C# 14](#)

```
extension<T>(IEnumerable<T>)
{ public static IEnumerable<T> Identity => []; }
```

no name: static members, operators · [C# 14](#)

```
extension(IEnumerable<int> s)
{
    public int this[int i] => s.ElementAt(i);
}
```

extension indexer · [C# 15 preview](#)

11 Collections & collection expressions

int[] numbers = [1, 2, 3]; collection expr · [C# 12](#)
List<string> names = ["A", "B"]; also sets, spans
int[] more = [0, .. numbers, 6]; spread operator
Dictionary<string,int> d = new(); no collection expr yet
d.TryGetValue("b", out int v) lookup without throwing
ReadOnlySpan<int> s = array; first-class spans · [C# 14](#)
numbers[^1] · numbers[1..^1] ^1 is last · [C# 8](#)
list[^1] works, but List<T> has no range slicing
HashSet<int> set = [1, 2, 3]; plus UnionWith etc.
queue.Dequeue() · stack.Pop() FIFO / LIFO
[with(capacity: 10), .. names] args · [C# 15 preview](#)

12 LINQ

xs.Where(p) · xs.Select(f) filter / project
.OrderBy(k).ThenByDescending(k) sort by several keys
people.GroupBy(p => p.City) gives IGrouping
nums.First(n => n > 5) also Any, All, Single
query.ToList() LINQ is lazy until materialized
from n in xs where n > 5 select n * 2 query syntax
nums.Chunk(3) · people.MaxBy(k) · [.NET 6](#), also MinBy
words.CountBy(w => w) · xs.Index() · [.NET 9](#)

```
people.LeftJoin(orders, p => p.Id, o => o.CustomerId,
    (p, o) => (p.Name, o?.Total));
```

· [.NET 10](#), keeps unmatched left rows
numbers.Shuffle() · [.NET 10](#), also Sequence
nums.Sum() · nums.Average() aggregates

13 Async

async Task<string> GetAsync() Task<T> returns a value
await Task.WhenAll(t1, t2); concurrent, wait for all
await Task.WhenAny(t1, t2); first one to finish
Task.WhenEach(tasks) as each lands · [.NET 9](#)
await Task.Delay(100, token); pass the token down
cts.CancelAfter(timeout); catch OperationCanceled
await foreach (var x in Xs()) IAsyncEnumerable · [C# 8](#)
.ConfigureAwait(false) in libraries, don't capture
async void event handlers only, cannot be awaited

14 Exceptions

```
try { Risky(); }
catch (IOException ex) when (IsTransient(ex)) { }
catch (Exception ex) { Log(ex); throw; }
finally { Cleanup(); }
```

when runs before unwinding; finally always runs
throw; · throw ex; keeps / resets the stack trace
throw new ArgumentException(m, nameof(x)); name the param
ArgumentNullException.ThrowIfNull(x); guard · [.NET 6](#)

```
class CustomerNotFound(int id) : Exception($"No {id}")
{ public int CustomerId { get; } = id; }
```

custom exception; no serialization ctor since .NET 8
int.TryParse(s, out int n) for expected failures

15 Code organization

namespace MyApp.Services; file-scoped · [C# 10](#)
global using System.Linq; whole project · [C# 10](#)
using static System.Math; then just Sqrt(x)
using Point = (int X, int Y); alias any type · [C# 12](#)
file class UserValidator { } this file only · [C# 11](#)

```
public // anywhere internal // this assembly
protected // derived private // this type
protected internal // assembly OR derived
private protected // assembly AND derived
```

access modifiers
partial string Conn { get; set; } partial prop · [C# 13](#)

```
#:package Spectre.Console@0.49.1
AnsiConsole.MarkupLine("[green>Hello[/]");
```

one .cs file is an app: dotnet run app.cs · [.NET 10](#)

16 New in C# 14 (.NET 10)

extension(IEnumerable<int> src) extension members
int H { get; set => field = v; } the field keyword
customer?.Order = value; null-conditional assign
nameof(List<>) unbound generics, gives "List"
ReadOnlySpan<int> s = array; first-class spans
(s, out n) => Try(s, out n) lambda modifiers, no types
public partial Settings(string p); partial ctors, events
public void operator +=(T x) compound assign in place

17 C# 15 preview (.NET 11)

union Pet(Cat, Dog, Bird); exactly one case type
closed record class Shape; exhaustive switch, no _
break outer; continue outer; labeled jumps
[with(capacity: 10), .. xs] coll. expr. arguments
this[int i] => s.ElementAt(i) extension indexer
Preview: needs <LangVersion>preview</LangVersion> on a .NET 11 preview SDK, and can still change.